**MBAi 460**
**Modern Software Architectures**
**Spring 2026**

# Project #01 – Part 03 (v1.0)

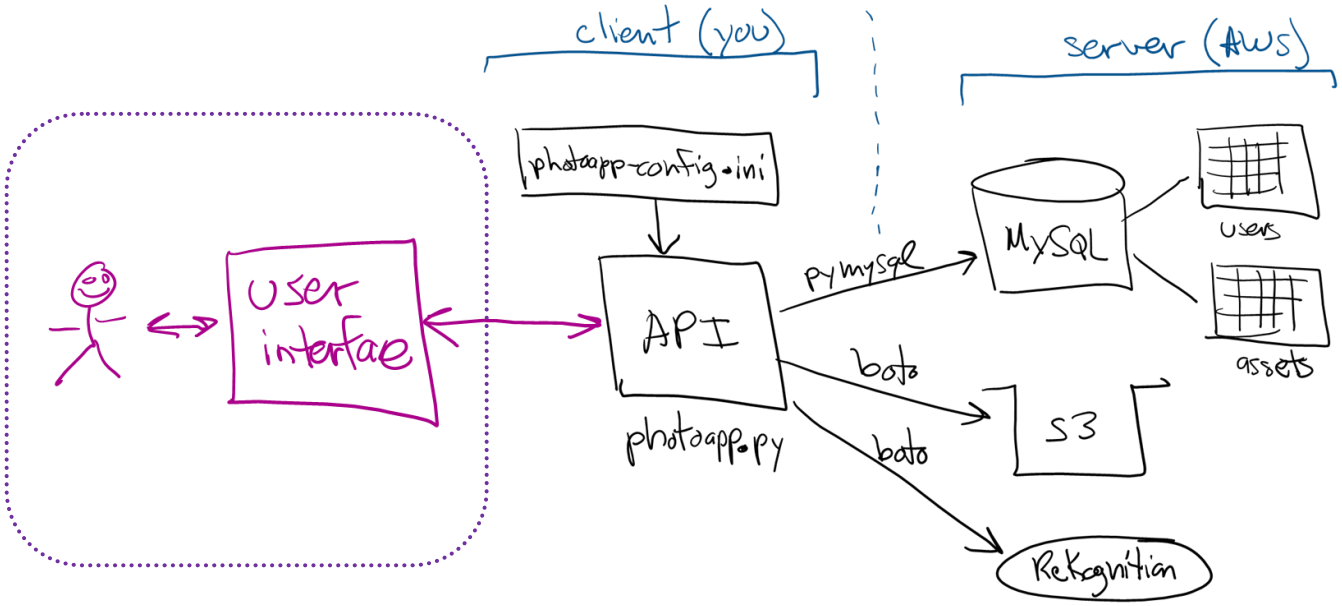
**Assignment:** PhotoApp with AWS S3, RDS and Rekognition
**Submission:** via Canvas
**Policy:** team-oriented, late work is accepted
**Complete By:** Monday April 27th @ 11:59pm CST

Late submissions: see syllabus for late policy… No submissions accepted after Thurs 04/30 @ 11:59pm

**Pre-requisites:** Project #01, Part 02 (API functions)

## Overview

In part 01 you built most of the AWS infrastructure needed for our ***photo-based application***, and in part 02 you implemented a set of API functions for accessing this infrastructure. What's left? A ***user interface*** to complete the application.



## Assignment requirements

Before we start, the idea of this assignment is less pressure, more fun. The infrastructure and the API functions were the cloud computing components and the focus of MBAi 460. The user interface is generally less interesting from an architectural or engineering standpoint, but UIs are critical to an application's success --- no one is going to use or provide funding for an app if the UI isn't modern and polished. UIs are not the focus of MBAi 460, but we need a UI so we're going to experiment with building one. We'll use this UI to complete project 01, and then use it again in project 02.

The assignment is intentionally very open-ended. But there are a few requirements:

1. *You may work alone, or in the same team from part 02.*
2. *Build a user interface (UI) using whatever technology you want; we'll give suggestions. You can build a console app, a Python graphical user interface (GUI), a web-based app, a mobile app, whatever you want to interface with your functions and AWS.*
3. *The UI must support the calling of all API functions. In a perfect world (but not required), it would nice if the UI had a way to display images that are downloaded, and provide a convenient way to select images for upload --- images can be listed by name, or thumbnail, or however you prefer. Even better if the app lists all the users in the database for easy selection when uploading, and lists all the images in the database for easy selection when downloading.*
4. *You are strongly encouraged to use an AI tool to help you design the UI and generate the code.*

## UI Frameworks

What UI framework should you use? Before we list some options, let's explain one important detail. GUIs require access to the graphics hardware on your computer so the GUI can be drawn on the screen. Docker images do not have access to your local graphics hardware, so the Docker image alone is not sufficient with most GUI frameworks. Here are four suggestions for how to proceed:

1. *Build a console-based app with no graphical user interface. The provided Docker image is sufficient. This is the least interesting approach.*
2. *Use the popular GUI framework https://nicegui.io which is already installed in your Docker image. When you run docker, use the command "./docker/run-8080". Then run your gui.py file using "python3 gui.py", and finally visualize your GUI by opening a web browser and browsing to http://localhost:8080 .*
3. *Install Python locally (i.e. outside of Docker) along with the required libraries pymysql and boto3, build a GUI, and simply run your app --- the GUI will naturally access your graphics hardware. This provides access to all the available GUI frameworks, not just nicegui.io.*
4. *If options 2 and 3 are not to your liking, the last option is to build a GUI using the provided Docker image, then connect the Docker image to a locally-running X server to handle the graphical display. Based on the GUI framework you choose, you may need to update your Docker file to install the necessary Python files and rebuild the Docker image.*

We recommend option #2 or #3; option #1 misses the point of the assignment, and option #4 requires the installation of X server software. If you already have experience with a particular GUI framework, or already have Python installed locally (and don't mind installing more Python libraries), option #3 gives you the most flexibility. If you like working with Docker and don't want to install more Python libraries on your laptop, option #2 offers an easy-to-learn approach that creates very nice, web-based GUIs.

Focusing on option #3, here is a list of the top-10 Python-based GUI frameworks. ***Tkinter*** is a popular choice because it's a standard, built-in Python library and builds cross-platform GUIs that run on Linux, Mac, and Windows. Here are 10 more frameworks, when in doubt go with #1 or #9:

1. https://customtkinter.tomschimansky.com, recommended by Prof. Bain and used in CS 110.
2. **PyQt/PySide:** These are Python bindings for the comprehensive Qt application development framework. They are known for their power, cross-platform compatibility, and ability to create professional, feature-rich applications with native look and feel. PySide is a good choice for commercial projects as it is available under the LGPL license, unlike the GPL option for PyQt.
3. **Kivy:** This open-source framework is designed for creating novel user interfaces, especially multi-touch applications for desktop and mobile platforms (iOS and Android). It excels in hardware-accelerated graphics and gesture handling.
4. **wxPython:** Known for its speed and stability, wxPython wraps the native widgets of the underlying operating system, allowing applications to blend seamlessly into different platforms (Windows, macOS, Linux). It offers a wide range of widgets for building complex, professional applications.
5. **PySimpleGUI:** Launched in 2018, PySimpleGUI is popular for its simplicity and ease of use, abstracting away the complexity of other frameworks like Tkinter and PyQt. It provides an intuitive interface, making it suitable for hobbyists and users who want to create simple GUIs quickly.
6. **PyGObject (GTK+):** This framework provides Python bindings for the GTK+ toolkit, which is widely used in Linux desktop environments like GNOME. It is a decent option for developers targeting a Linux-first audience.
7. **BeeWare Toga:** Toga is part of the BeeWare project, which aims to provide truly cross-platform, native-looking GUIs on every operating system. Its goal is to allow writing code once and deploying it everywhere with a native experience.
8. **DearPyGui:** A rising star known for its simplicity and outstanding performance, DearPyGui is based on Dear ImGui and is suited for high-performance applications that may require custom rendering.
9. **Streamlit:** While technically a data app framework rather than a traditional GUI framework, Streamlit is the go-to choice for quickly turning Python scripts into interactive web applications and dashboards, particularly in the data science community.
10. **PyForms:** Built on top of PyQt, PyForms offers a modular approach to GUI development, simplifying the process of creating form-based applications

<< continued on next page >>

## Deliverables

There are three deliverables for this part of the project, all on Canvas; there is nothing to submit on Gradescope. The deliverables are:

1. *Create a video recording that demonstrates your UI, executing all the API functions. The recording doesn't have to be long, 5-10 mins should be plenty of time. We don't need to see your face, just your voice and screen capture of the UI app running. When you video is done, post it and create a PUBLIC link that does not require a login/password. Then login to Canvas and paste the link to your video in the assignment "Project 01, part 03 – Client". If you worked in a team, there should be only one submission for the team.*

2. *Create a .zip file that contains your UI implementation, and submit to Canvas under the assignment "Project 01, part 03 – implementation". If you worked in a team, there should be one submission for the team.*

3. *Submit a paragraph or two that describes what UI framework you used, and any instructions that are needed if we want to run your GUI. Also, reflect on the AI usage: what prompt(s) did you give? How was the experience, e.g. how well did it work? Copy-paste your text into the Canvas assignment "Project 01, part 03 – reflection". If you worked in a team, there should be one submission for the team.*

That's it, have fun!